13: Review

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Some material taken from EECS370 at U. of Michigan

Announcements

- P3: Extended a week.
- P4: Count the number of binary 1's in a file.
  - Should be out this week.
- Exam Wednesday (3/3)
  - Review on Monday

Exam Details

- 5 Questions
  - Multiple sub-questions

- Some short answer
- Some fill-in-the blank/code/table

Topics

- Major (1 Question / Topic)
  - MMIO
  - Bus Interfaces
  - DMA

- Minor (2 Questions)
  - Memory Protection
  - Caches
  - Virtual Memory

A “Cheat” Sheet is Allowed

- 1-sided
- 8.5”x11” paper
- Handwritten (not photocopied)

- Keep this, you can use the other side for the 2nd exam.

MMIO

OLD: IO pretends to be CPU register
New IO pretends to be memory address

Memory-Mapped Input/Output

- What is MMIO?
- Why do we use it?

Interacting w/ peripherals using loads & stores

- scales to larger # of peripherals w/o needing special instructions

MMIO Loads

- In ASM?

```
mov r2, 0x40000000 ;
ldr r3, [r2, 0x144];
```

- In C?

```
# define ADDR 0x40000144
volatile uint32_t * addr = (uint32_t *)(ADDR);
uint32_t x = * addr;
```

Bus Interfaces

- AXI4 “Full” vs. AXI4 Lite vs. AXI4 Stream

- How are they different?
- Where do we use them?

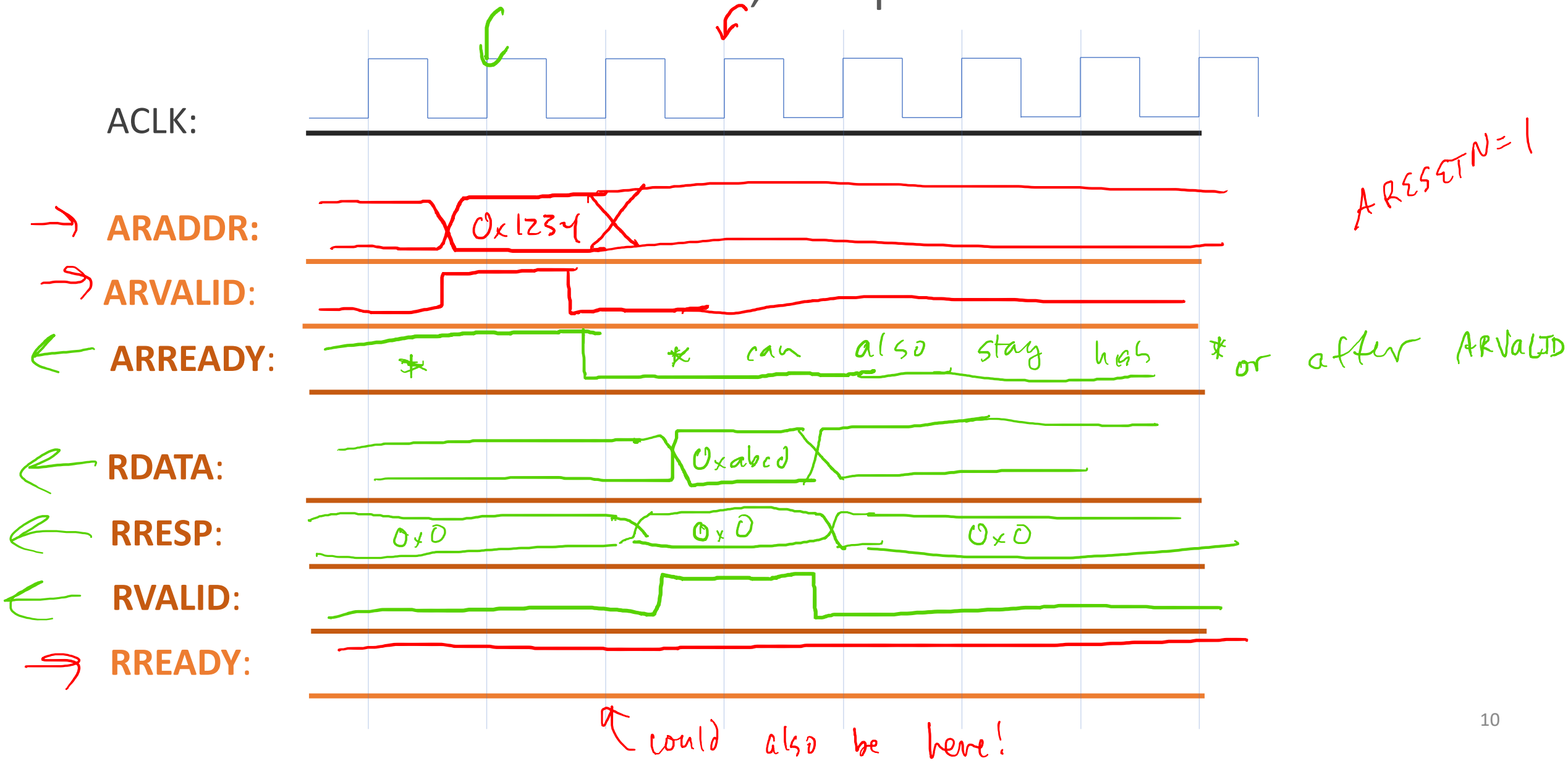
AXI Full: we don't!
AXI Lite: control, infrequent use
AXI stream: Bulk data processing

AXI
fast
complex
mmIO

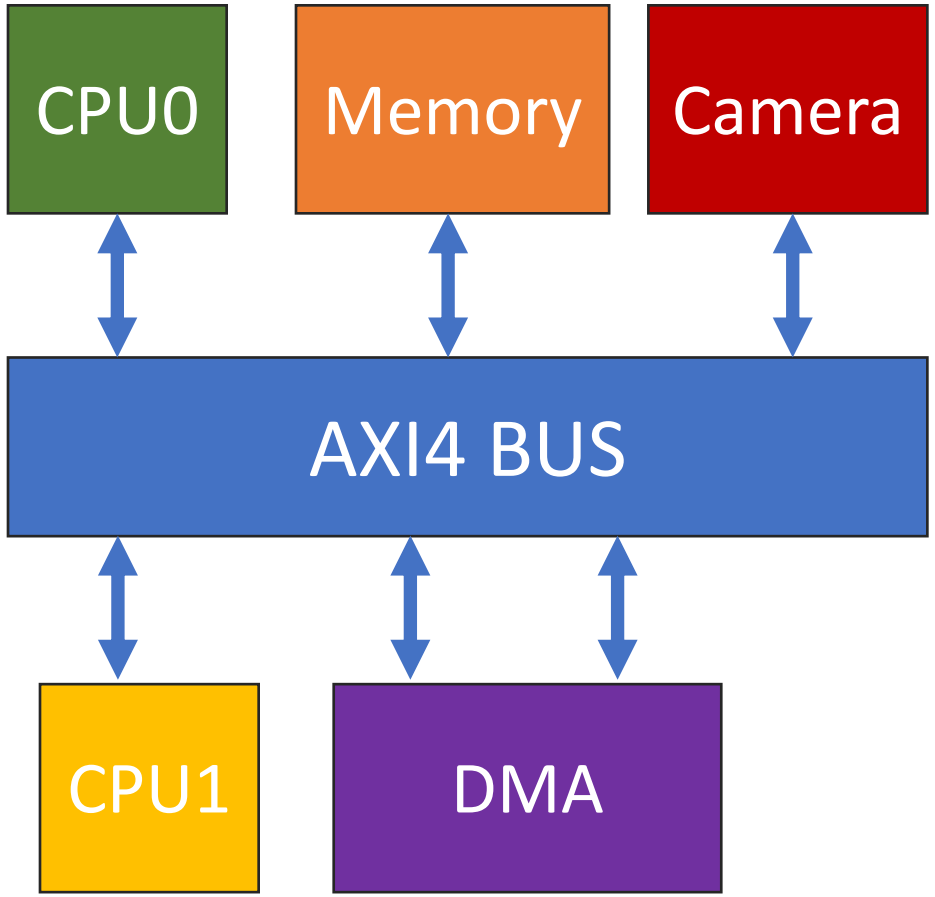
AXI Lite
slower
simpler
mmIO

AXI Stream
fast
simpler
non-mmIO

AXI4Lite: Load 0x1234, response: 0xabcd



DMA



- What is DMA?
- What's the goal of DMA?

mini-cpu for copy

DMA: the copy cpu

```
void dma (uint32_t * from,
          uint32_t * to,
          uint32_t size)
{
    reg x;
    for( int i=0; i < size, ++i){
        x= from[i];
        too[i]=x;
    }

}
```

DMA Control Design

```
void dma (uint32_t * from,
          uint32_t * to,
          uint32_t size)
{
   register uint32_t reg;

   for (int i = 0; i < size; ++i){

      reg = from[i]; //load

      to[i] = reg; //store
   }
}
```

- What interfaces do you need?
- How do you start/stop DMA?
- Design a DMA state machine?

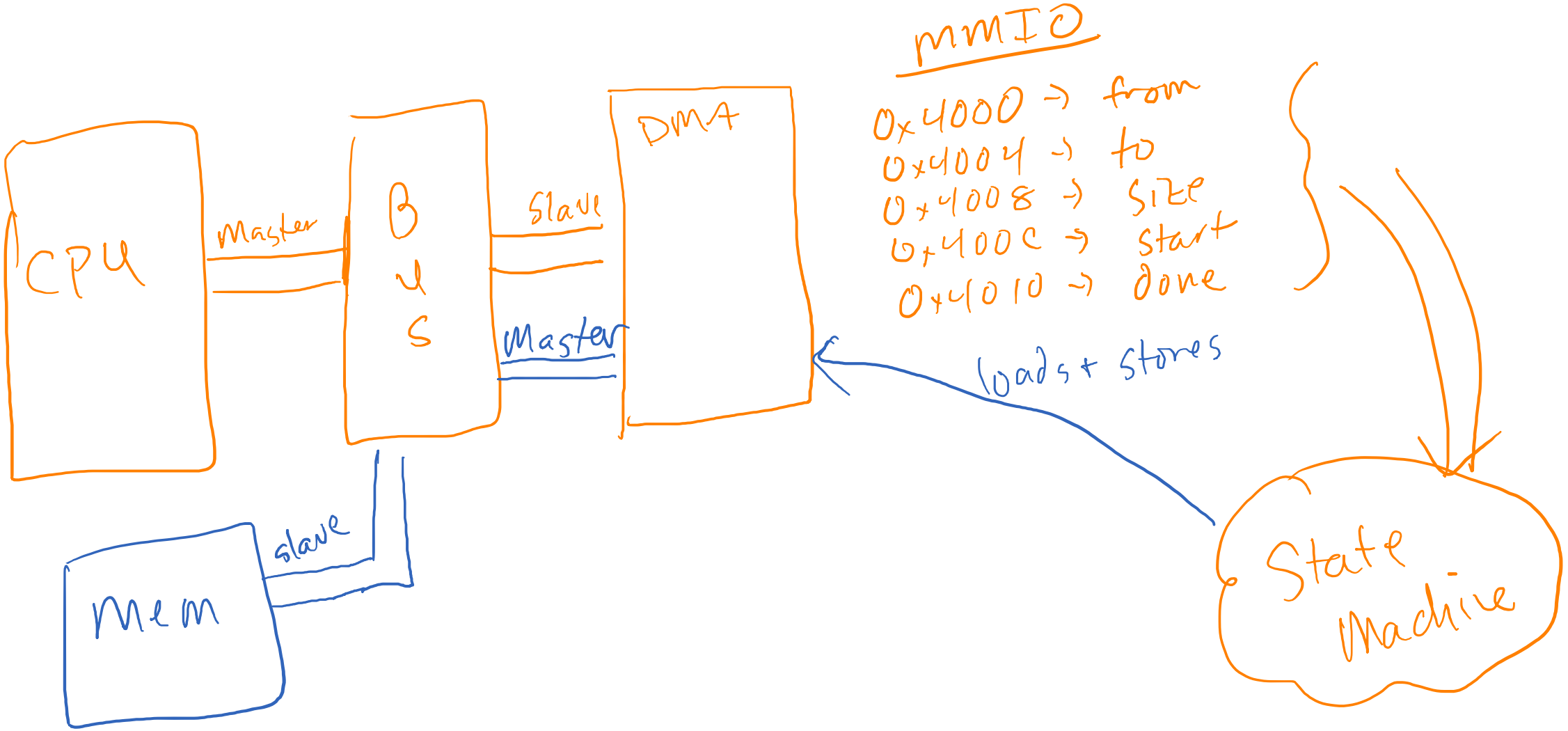
DMA Control

```
void dma (uint32_t * from,
            uint32_t * to,
            uint32_t size)
{
   register uint32_t reg;

   for (int i = 0; i < size; ++i){

      reg = from[i]; //load

      to[i] = reg; //store
   }
}
```

- AXI4 Master Interface
  - Loads + Stores
- 5 MMIO registers
  - Control (Start)
  - Status (Done)
  - Source (`from`)
  - Destination (`to`)
  - `size` (in Bytes)

*future exam question: Build own DMA ctrlr.

DMA Control Interface



Memory Protection

- What is memory protection? prevent unauthorized loads & stores
- Why is it useful? → protect memory from accidental or malicious access
- When can it be a problem?

MPU Table

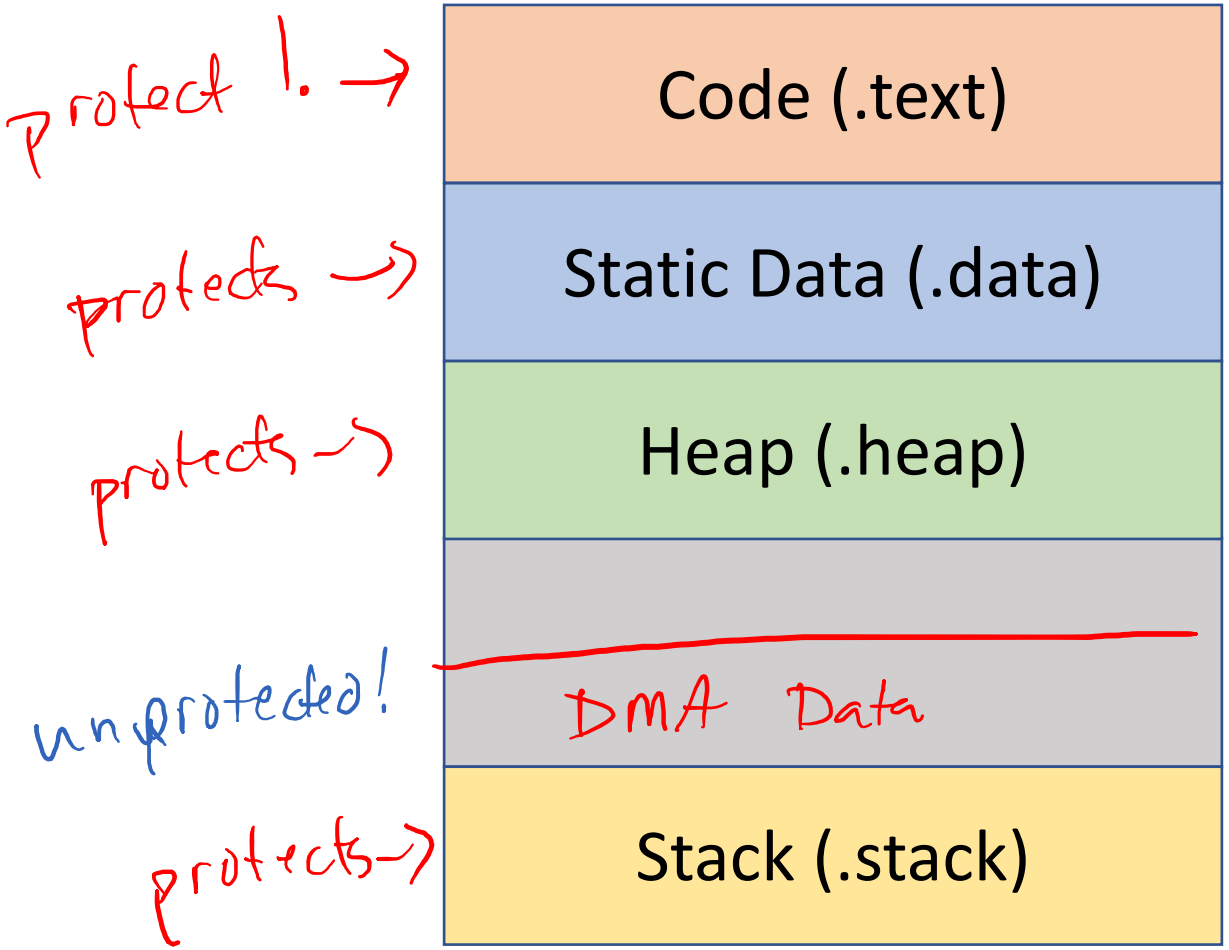
Covers 0x1200 – 0x12ff

OS Mode

BASE Addr	Offset	Source	Permission
0x1200	0xff	CPU0	Read, !write
"Code"	sizeof(code)	CPU	Read, !write
"data"	sizeof(data)	CPU	Read, Write
"data"	"	DMA	Read, !write
"DMAdata"		CPU	R, W
"DMAdata"	"	DMA	R, W

MMIO

MPU Workaround?



Translation Look-aside Buffer (TLB)

Virtual Address

32-bit

Virtual Base	Offset

20 bits

12-bits

0x 12345 | abc

Virtual Base

v	~~tag~~	Physical Addr
V	12344	
V	12345	ABCDE
V	12346	

0xABCDE | ABC

32-bits

Physical Base	Offset
20	12

Physical Address

Caches and TLBs

Processor

0001 Ld R1 ← M[1]
0111 Ld R2 ← M[7]
0000 St R2 → M[0]
0110 St R1 → M[6]
1010 Ld R2 ← M[10]

R0	
R1	21
R2	29
R3	

TLB

Virt	Base
3 bits	1 bit

V	Tag	Physical
1	0	4
1	3	0

Virt Base ⇒ Phys Base

000 1 → 100 1
011 1 → 000 1
000 0 → 100 0
011 0 → 000 0
101 0

Cache (Physical)

tag = 0

V	d	tag	data
~~0~~	1	4	~~16~~ 29
			21
0	1	0	~~X~~ 21
			29

* Assume Least-Recently used replacement

Memory

0000	0	78
0001	1	29
0010	2	120
0011	3	123
0100	4	71
0101	5	150
0110	6	162
0111	7	173
1000	8	18
1001	9	21
1010	10	33
1011	11	28
1100	12	19
1101	13	200
1110	14	210
1111	15	225

Caches and TLBs

Processor

Ld R1 ← M[1]
Ld R2 ← M[7]
St R2 → M[0]
St R1 → M[6]
Ld R2 ← M[10]

R0	
R1	
R2	
R3	

TLB

V	Tag	Physical
1	0	4
1	3	0

Cache (Physical)

V	d	tag	data
0			
0			

Memory

0	78
1	29
2	120
3	123
4	71
5	150
6	162
7	173
8	18
9	21
10	33
11	28
12	19
13	200
14	210
15	225

13: Review

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University